Лабораторна робота №10

Тема: React та REST API: API layer, Repository Pattern, асинхронні операції та CRUD.

Мета: навчитися інтегрувати React-застосунок із REST API та відокремлювати UI від деталей HTTP-запитів. Опанувати асинхронні стани, CRUD, пагінацію, скасування запитів та спрощений Repository Pattern.

Вимоги до звіту

1. Використовувати React 19.x та Node.js 22.22 або новішу версію.
2. Використовувати TypeScript.
3. Роботу з HTTP винести в окремий API layer.
4. Реалізувати Repository для операцій над `Task`.
5. Реалізувати GET, POST, PUT/PATCH та DELETE.
6. Реалізувати отримання списку з пагінацією.
7. Реалізувати перегляд деталей окремого `Task` через GET за `id`.
8. Для асинхронних операцій передбачити loading, success, error та empty states.
9. Для GET-запитів передбачити можливість скасування попереднього запиту через `AbortController` або еквівалентний механізм обраного HTTP-клієнта.
10. Після помилки завантаження користувач повинен мати можливість повторити запит.
11. Використати `fetch` або Axios — вибір залишається за студентом.
12. API URL отримувати з environment variable, наприклад `VITE_API_URL`.
13. Додати файл `.env.example` без реальних секретів.
14. UI-компоненти не повинні містити URL endpoint-ів або прямі HTTP-виклики.
15. Не використовувати `alert`.
16. Для стилізації використати один із підходів: SCSS із дотриманням BEM, Bootstrap або Tailwind CSS.
17. Якщо обрано SCSS, дотримуватися BEM-патерну для класів.
18. Дотримуватися ESLint, TypeScript та Conventional Commits.

Теоретична частина

Опрацювати:

1. Fetch API.
2. AbortController.

3. Axios.
4. Матеріали лекції щодо React + REST API, asynchronous state та CRUD.
5. Поняття Repository Pattern та separation of concerns.
6. DummyJSON Todos.

Практична частина

Стилізація та BEM

Для стилізації обрати один із трьох підходів:

1. **SCSS + BEM** — власні стилі з класами за схемою Block__Element—Modifier;
2. **Bootstrap** — готові utility та component класи;
3. **Tailwind CSS** — utility-first класи.

Не потрібно поєднувати всі три підходи одночасно.

Якщо обрано **SCSS + BEM**:

- стилі організувати за компонентами або логічними блоками;
- кожен компонент або логічний UI-блок має власний `.scss` файл поруч із відповідним компонентом;
- не зберігати стилі компонентів у загальному `App.css` або іншому великому спільному файлі;
- назви класів повинні відповідати BEM;
- не прив'язувати стилі до глибокої структури DOM;
- глобальні стилі залишати лише для справді глобальних речей: `body`, базові стилі, шрифти, reset тощо.

Git workflow

Рекомендується виконувати роботу у feature-гілці `feature/rest-api`.

Приклад Conventional Commit:

```
feat(lab-10): integrate task repository with REST API
```

1. Конфігурація API

Замінити локальне джерело даних Study Planner на REST API.

Для лабораторної роботи рекомендується використовувати DummyJSON з ресурсом `todos`:

https://dummyjson.com/todos. За погодженням з викладачем можна використати інший REST API з аналогічною функціональністю.

Створити:

```
.env
.env.example
```

У `.env.example` вказати лише приклад:

```
VITE_API_URL=https://dummyjson.com
```

Файл `.env` не додавати до Git.

API, що використовується в роботі

Для базового варіанта використовуйте DummyJSON:

Метод	Endpoint	Призначення
GET	`/todos?limit=10&skip=0`	список завдань з pagination;
GET	`/todos/:id`	отримання одного завдання;
POST	`/todos/add`	створення завдання;
PUT/PATCH	`/todos/:id`	оновлення завдання;
DELETE	`/todos/:id`	видалення завдання.

Відповідь списку має вигляд:

```
type TodosResponse = {
  todos: ApiTodo[];
  total: number;
  skip: number;
  limit: number;
};
```

Важливе уточнення щодо навчального API

DummyJSON симулює POST, PUT/PATCH та DELETE: відповідь повертається, але зміни не записуються назавжди на сервері.

Після успішної mutation-операції студент повинен самостійно оновити client state/UI. Створювати власний backend для цієї лабораторної роботи не потрібно.

Це обмеження є частиною навчального сценарію: воно дозволяє окремо продемонструвати HTTP-операцію та оновлення стану React-застосунку.

2. API model та domain model

Модель, яку повертає API, не обов'язково повинна збігатися з моделлю, яку використовує UI.

Для DummyJSON todo має вигляд:

```
type ApiTodo = {
  id: number;
  todo: string;
  completed: boolean;
  userId: number;
};
```

У Study Planner можна залишити власну domain-модель `Task` з лабораторної №8.

Repository повинен виконувати mapping між API DTO та domain model, якщо це необхідно.

Наприклад:

```
function toTask(todo: ApiTodo): Task {
  return {
    id: todo.id,
    title: todo.todo,
    description: '',
    completed: todo.completed,
    priority: 'medium',
    category: 'General',
  };
}
```

Поля `description`, `priority` та `category` у такому варіанті не підтримуються DummyJSON і можуть залишатися client-only metadata. Не потрібно вигадувати API-поля, яких немає в контракті. Обмеження навчального API не є помилкою.

3. HTTP client

Створити окремий модуль, наприклад:

```
src/
└──── api/
    └──── httpClient.ts
```

Він повинен:

- будувати URL із `VITE_API_URL`;
- виконувати HTTP-запити;

- перевіряти `response.ok` або еквівалентний статус;
- повертати дані або кидати помилку, яку може обробити вищий рівень.

Компоненти React не повинні знати про реалізацію HTTP-клієнта.

4. Repository Pattern

Створити окремий repository для задач:

```
src/
└──── repositories/
    └──── taskRepository.ts
```

Repository використовується тут як навчальний приклад separation of concerns. У невеликому застосунку можна реалізувати data access простіше, але в цій лабораторній repository потрібен саме для тренування розділення UI та API-логіки.

Для DummyJSON repository повинен приховувати endpoint-и та DTO-формат.

Repository повинен надавати методи приблизно такого вигляду:

```
getAll()
getById(id)
create(task)
update(id, task)
delete(id)
```

Назви методів можна змінити, якщо вони зберігають ту саму відповідальність.

Компоненти повинні працювати з `taskRepository`, а не будувати URL endpoint-ів і не дублювати HTTP-логіку.

Не потрібно створювати складну універсальну систему repository для всього застосунку. Достатньо одного зрозумілого repository для `Task`.

5. Завантаження списку та пагінація

При відкритті Study Planner:

1. показати loading state;
2. викликати `taskRepository.getAll()`;
3. відобразити отримані задачі;
4. у разі помилки показати зрозуміле повідомлення;
5. дати користувачу можливість повторити запит;
6. дозволити перейти на наступну та попередню сторінку;

7. показувати номер поточної сторінки або інший зрозумілий індикатор позиції.

Для DummyJSON можна використовувати `limit` і `skip`.

Під час переходу між сторінками попередній GET-запит потрібно скасувати, якщо він ще виконується.

6. Loading, error, empty та success

Реалізувати окремі візуальні стани:

```
Loading...
Помилка завантаження. [Спробувати ще]
Завдань не знайдено.
```

Порожній результат не повинен виглядати як зламаний інтерфейс.

7. Деталі завдання

При виборі завдання відкрити його деталі та отримати актуальні дані через `taskRepository.getById(id)`.

У деталях показати щонайменше назву, опис, категорію, priority та completed.

Дані для деталей не потрібно дублювати в окремому глобальному Context: достатньо локального state компонента або іншого простого client-side state.

8. Створення завдання

Форма з лабораторної №8 повинна створювати нове завдання через `taskRepository.create()`.

Після успішного створення:

1. оновити список у UI;
2. очистити форму;
3. показати короткий success state або inline-повідомлення.

9. Оновлення та видалення

Для існуючого завдання реалізувати:

- перемикання `completed`;
- редагування назви;
- видалення.

Операції повинні проходити через repository.

Для видалення додати зрозуміле підтвердження в інтерфейсі.

10. Скасування запитів

Для списку та деталей використовувати асинхронний запит із можливістю скасування.

Приклад для `fetch`:

```
const controller = new AbortController();

fetch(url, { signal: controller.signal });

return () => controller.abort();
```

Якщо використовується Axios, скористайтеся його підтримкою `AbortSignal`. Скасований запит не повинен показуватися користувачу як звичайна помилка завантаження.

11. Архітектура стану

Розділити:

- **server state** — дані, отримані через API;
- **client/UI state** — фільтр, поточна сторінка, стан форми, selected task тощо.

Поточний localStorage із лабораторної №9 не потрібно використовувати як друге джерело правди для server state. Якщо потрібне локальне збереження, його можна залишити лише для client/UI state.

12. Збереження коду з лабораторної №9

Context, reducer та Custom Hooks із попередньої лабораторної можна зберегти там, де вони справді корисні. Не потрібно переносити server state у `TaskContext` лише для того, щоб зберегти архітектуру №9. Дані `Task`, які приходять із server state, не повинні дублюватися без потреби в кількох незалежних state.

13. Додаткові завдання

1. Додати пошук із debounce.
2. Реалізувати optimistic update для зміни `completed`.
3. Додати Axios instance з базовим URL.
4. Додати централізовану нормалізацію помилок.

Рекомендована структура

```
src/
 ├──── api/
 │     └──── httpClient.ts
 ├──── repositories/
 │     └──── taskRepository.ts
 ├──── components/
 ├──── context/
 ├──── hooks/
 ├──── types/
 ├──── pages/
 ├──── App.tsx
 └──── main.tsx
```

Структура може бути змінена, але межа між UI, HTTP та repository повинна залишатися очевидною.

Демонстрація на захисті

За час захисту необхідно продемонструвати:

1. завантаження списку із REST API та пагінацію;
2. відкриття деталей окремого завдання;
3. створення нового завдання;
4. одну операцію оновлення або видалення;
5. loading/error/empty/success state;
6. у коді показати `httpClient`, `taskRepository` та скасування GET-запиту.

Не потрібно демонструвати всі додаткові можливості.

Контрольні запитання

1. Що таке REST API?
2. Чим GET, POST, PUT/PATCH та DELETE відрізняються один від одного?
3. Чому API-логіку доцільно винести з React-компонентів?
4. Що таке Repository Pattern?
5. Яку відповідальність має repository у цій лабораторній?
6. Чим API client відрізняється від repository?
7. Які стани потрібно врахувати для асинхронного запиту?
8. Що таке server state і чим він відрізняється від client/UI state?

9. Чому не варто дублювати server state у localStorage без потреби?
10. Для чого потрібен `AbortController`?
11. Що станеться, якщо компонент завершить існування під час виконання запиту?
12. Чим fetch відрізняється від Axios?
13. Що таке optimistic update?
14. Чому після успішного POST не обов’язково виконувати повне перезавантаження сторінки?
15. Як правильно обробити HTTP-помилку?